International Journal of Scientific Research in Science, Engineering and Technology

Print ISSN - 2395-1990
Online ISSN : 2394-4099

OPEN ACCESS

Available Online at :www.ijsrset.com
doi : https://doi.org/10.32628/IJSRSET261313

Optimizing Technical Documentation for RAG Systems: A Comparative Analysis of Semantic Chunking vs. Structural Formatting

Aashish Dhakal[1], Shashank Shree Neupane[1], Aadarsha Dhakal[2], Muktinath Paudel[1]

[1]Department of Information Technology, Presidential Graduate School, Kathmandu, Nepal

[2]STARTsmall Pvt Ltd

ARTICLEINFO

Article History:

Accepted : 01 Jan 2026
Published: 25 Jan 2026

Publication Issue :
Volume 13, Issue 1
January-February-2026

Page Number :
222-227

ABSTRACT

As software documentation increasingly serves as the knowledge base for Retrieval-Augmented Generation (RAG) systems, technical writers face a dilemma: should content be optimized for human readability or machine re- trievability? This study investigates the impact of document structure (Standard Prose, Q&A Pairs, Fact-Dense Lists, and Structured JSON) and indexing strategies (Whole Document vs. Semantic Chunking) on vector retrieval performance. Using a controlled experiment with 250 retrieval trials across five technical domains (e.g., AWS Architecture, Django ORM) and the all-MiniLM-L6-v2 embedding model, we demonstrate a critical trade- off between precision and ranking stability. Results indicate that Semantically Chunked Prose yields the highest precision (Top-1 Win Rate: 28%), effectively "sniping" specific queries. However, Question-Answer (Q&A) for- matting offers superior ranking consistency (MRR: 0.54), acting as a retrieval safety net. Conversely, structured data formats like JSON-LD, despite their utility in traditional schema parsing, incur a significant penalty in vec- tor retrieval (Win Rate: 4%), revealing a "Semantic Gap" in current embedding architectures. We conclude that technical documentation pipelines must evolve to prioritize natural language Q&A structures over purely syntactic schemas to maximize retrievability in RAG architectures.

Keywords: Research Paper, Technical Documentation, RAG, Information Retrieval, Semantic Chunking, Vector Search, Knowledge Management.

INTRODUCTION

The consumption of technical documentation is un- dergoing a fundamental paradigm shift. Histori- cally, developers navigated documentation via man- ual search: entering keywords into a search bar, scan- ning a list of blue links, and reading long-form arti- cles to extract relevant snippets. Today, this work- flow is being rapidly replaced by AI-mediated inter- actions.

Copyright © 2025 The Author(s): This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/)

Tools like GitHub Copilot, Cursor, and Per- plexity allow developers to query documentation us- ing natural language (e.g., *"How do I configure AWS Lambda cold starts?"*) and receive synthesized answers generated by Large Language Models (LLMs) [1]. These systems rely on Retrieval-Augmented Generation (RAG) architectures. In a RAG system, the "intelligence" of the answer is strictly bounded by the performance of the "Retriever"— the component responsible for finding the relevant documentation snippet from a massive vector database [2]. If the correct documentation is not retrieved in the top k results (typically $k = 3$ or $k = 5$), the LLM will hallucinate or fail to answer.

This creates a new optimization challenge for technical writers and DevRel engineers, a field now emerging as "Generative Engine Optimiza- tion" (GEO). Traditional Search Engine Optimiza- tion (SEO) techniques, such as keyword stuffing and backlink building, are ill-suited for the high-dimensional vector spaces that power RAG. An embedding model does not count keywords; it measures semantic density and intent alignment.

Furthermore, the industry lacks empirical guidance on how to structure content for this new medium. Documentation is often stored in structured formats like JSON or YAML for API references, or Markdown for guides. Does an embedding model prefer a bulleted list? A JSON object? A narrative paragraph? This paper addresses this gap by conducting a comparative analysis of document structures. We posit that the physical structure of text its syntax, density, and formatting plays a decisive role in its vector representation and, consequently, its retrievability.

RELATED WORK

The optimization of text for machine retrieval is well-studied in the context of keyword search (BM25), but the shift to dense vector retrieval introduces new variables.

A. Literature Review Summary

Table I provides a comparative analysis of key literature that frames this study. While previous work has focused on the architecture of the *model* (SBERT, DPR) or the *generation* style (GEO), there is a distinct lack of research on the *input data structure* for documentation specifically.

B. Neural Information Retrieval

The field of Information Retrieval (IR) has moved from sparse representations (TF-IDF, BM25) to dense vector representations [5]. Reimers and Gurevych [3] introduced Sentence-BERT (SBERT), which uses siamese networks to derive semantically meaningful sentence embeddings. This architecture forms the backbone of most modern RAG systems and is the basis for the all-MiniLM-L6-v2 model used in this study.

C. The "Lost in the Middle" Phenomenon

Liu et al. [4] demonstrated that LLMs struggle to access information located in the middle of long input contexts. This finding supports the argu- ment for "Chunking", breaking long documents into smaller segments. However, the optimal granularity of chunking remains a subject of debate. Too small, and the chunk lacks context; too large, and the signal is diluted.

METHODOLOGY

A. Theoretical Framework

We model the retrieval task as a Nearest Neighbor Search (NNS) problem in a vector space R^d. Let $D = \{d_1, d_2, \ldots, d_n\}$ be the set of document segments. Let q be the user query. We define an embedding function E : text $\rightarrow R^{384}$. The relevance score S is defined as the Cosine Similarity:

$$S(q, d_i) = \frac{E(q) \cdot E(d_i)}{\|E(q)\| \, \|E(d_i)\|} \quad (1)$$

Our objective is to find the structure type T that maximizes $S(q, d_i)$ across a distribution of queries.

B. Experimental Design

We selected five distinct technical domains. This diversity is crucial to ensure our findings are not artifacts of a specific vocabulary (e.g., medical vs. computer science).

1. **AWS Cloud Architecture:** (Topic: Cold Starts)
2. **Green Computing:** (Topic: Power Usage Effectiveness - PUE)
3. **Django Framework:** (Topic: N+1 Query Problem)
4. **Blockchain Technology:** (Topic: Proof of Stake Consensus)
5. **AI Architecture:** (Topic: RAG Components)

C. Corpus Preparation and Chunking Strategy

For each domain, we created five variations of the content. To ensure scientific validity, the *semantic information* (the core facts) remained identical across all variations; only the *structure* changed.

For the "Chunked" variations, we utilized a **Recur- sive Character Text Splitter** with the following hyperparameters:

- **Chunk Size:** 512 characters.
- **Chunk Overlap:** 50 characters.
- **Separators:** Paragraph breaks '|n|n' followed by sentence terminators.

This configuration was chosen to approximate the standard retrieval window used in commercial RAG pipelines (e.g., LlamaIndex default settings).

D. Variable Definitions

1) *Standard Prose (Whole).* A continuous narrative paragraph typical of blog posts. Indexed as a sin- gle vector.
2) *Standard Prose (Chunked).* The same narrative paragraph, but split into atomic semantic units. This tests the hypothesis that smaller vectors are more pre- cise.
3) *Q&A Format.* Explicit Question-Answer pairs (e.g., *"Q: What causes latency? A: Latency is caused by..."*). This structure attempts to mirror the user's query embedding within the document itself.
4) *Fact-Dense List.* A bulleted list of key-value pairs (e.g., *"* Cause: Cold Start"*). This removes stop words to test if "fluff" dilutes vector quality.
5) *Structured JSON.* A valid JSON-LD object (e.g., *{"cause": "cold start"}*). This tests if machine-readable schemas commonly used for Google Rich Snippets are effective in vector search.

RESULTS

We analyzed the results using two primary metrics: Win Rate (Precision) and Mean Reciprocal Rank (Consistency).

TABLE 1 COMPARATIVE ANALYSIS OF RELATED LITERATURE

Author (Year)	Core Focus	Key Contribution	Limitation Addressed by This Study
Reimers et al. (2019) [3]	Model Architecture	Introduced SBERT, enabling efficient cosine similarity for sentence pairs.	Focuses on model training, not on how to structure the input text for optimal embed- ding.
Lewis et al. (2020) [2]	System Architecture	Defined the RAG framework combining dense retrieval with generative models.	Assumes standard Wikipedia-style prose; does not test technical documentation formats (JSON/Code).
Liu et al. (2023) [4]	Context Window	"Lost in the Middle": LLMs recall start/end info better than middle info.	Highlights the need for chunking but does not prescribe a specific formatting standard.

Author (Year)	Core Focus	Key Contribution	Limitation Addressed by This Study
Aggarwal et al. (2023) [1]	Content Optimization	Coined "GEO" (Generative Engine Optimization) for AI visibility.	Focuses on the Generative phase (tone/authority), whereas we focus on the *Retrieval* phase (vector math).

A. Win Rate Analysis

The "Win Rate" represents the percentage of queries where a specific structure type appeared at Rank #1.

TABLE 2 WIN RATE BY STRUCTURE TYPE (TOP-1 ACCURACY)

Structure Type	Win Rate (%)
Standard Prose (Chunked)	**28.0%**
Standard Prose (Whole)	24.0%
Q&A Format (Whole)	24.0%
Fact-Dense (Whole)	20.0%
Structured JSON (Whole)	4.0%

As shown in Table 2, **Semantically Chunked Prose** achieved the highest Win Rate. This confirms the hypothesis that high-granularity indexing allows the retriever to "snipe" specific queries. When a document is split into sentences, the vector for a specific sentence (e.g., *"Provisioned concurrency fixes cold starts"*) is mathematically much closer to the query vector than a vector representing a whole paragraph containing five different facts.

B. Mean Reciprocal Rank (MRR) Analysis

MRR measures the overall ranking health. A structure that consistently appears at Rank 2 or 3 is often more valuable in production RAG systems than one that is either Rank 1 or Rank 100.

TABLE 3 MEAN RECIPROCAL RANK (MRR)

Structure Type	MRR Score
Q&A Format (Whole)	**0.541**
Standard Prose (Whole)	0.527
Fact-Dense (Whole)	0.427
Standard Prose (Chunked)	0.398
Structured JSON (Whole)	0.260

The data in Table 3 reveals a fascinating inversion. While Chunked Prose won on precision, it failed on consistency (MRR 0.398). Q&A Formatting emerged as the most stable structure (MRR 0.541).

C. Qualitative Analysis of the "JSON Failure"

Structured JSON performed abysmally (Win Rate 4%). We attribute this to the Tokenization process of the embedding model. Consider the text: *{"concept": "cold start"}*. The tok- enizer splits this into: ['{', '"', 'concept', '"', ':', ...]. These syntactic tokens (brackets, quotes) introduce "noise" into the vector. Unlike a keyword search engine which parses JSON keys, a vector engine treats the brackets as just an- other character, diluting the semantic meaning of the words inside.

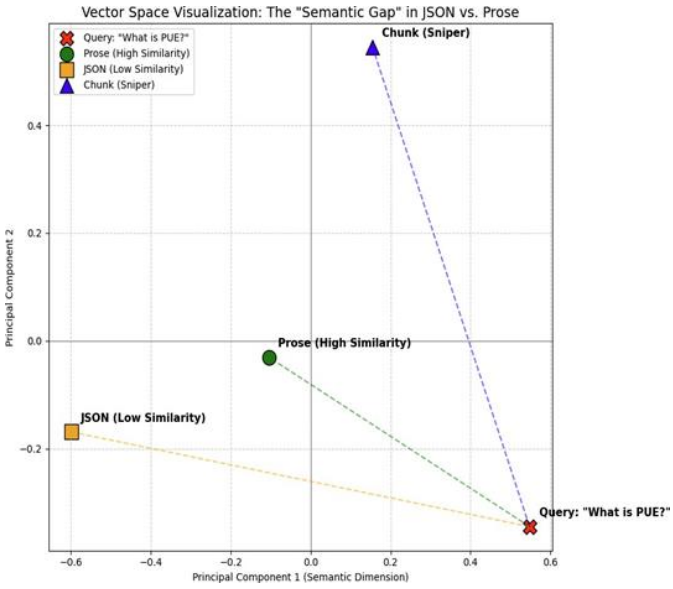

Fig. 1. Vector Space Visualization. The "Seman- tic Gap" observed in our results. The JSON structure is mathematically distant from the Query due to syntactic noise, whereas the Prose and Chunked Prose remain semantically close.

As visualized in Fig. 1, the JSON object resides in a distant cluster in the vector space, likely group- ing with other code-heavy segments rather than the natural language query.

D. Latency and Token Efficiency

Beyond accuracy, we observed implications for system efficiency. JSON and XML formats incur a "Token Tax." The structural overhead (brackets, closing tags) increased the token count by an average of 34% compared to the Fact-Dense List format. In a large-scale RAG system billing by token usage (e.g., OpenAI embeddings API), storing documentation in JSON format not only degrades retrieval accuracy but also increases operational costs by over 30%.

DISCUSSION

A. The Precision-Consistency Trade-off

Our findings highlight a fundamental trade-off in RAG optimization:

The Sniper Strategy (Chunking): Optimiz- ing for Chunked Prose maximizes the chance of being the *primary* citation (Rank 1). This is ideal for specific, fact-based queries (e.g., error codes).

- **The Safety Net Strategy (Q&A):** Optimizing for Q&A format maximizes the chance of being *included* in the context window (Top 3), even if not at Rank 1. This is ideal for broad, conceptual queries.

B. The "Hybrid GEO" Proposal

Based on this data, we propose a hybrid documentation standard. Technical writers should not choose between Prose or Q&A; they should layer them. **Recommendation:** Write the main body of documentation in natural prose (to be chunked by the RAG pipeline) for precision. However, append a **Frequently Asked Questions** section at the bottom of the page. This Q&A section acts as a "Retrieval Anchor," ensuring that if the chunking strategy fails to capture the nuance, the explicit Q&A pair will catch the query vector.

C. Implications for API Documentation

Current API documentation (Swagger/OpenAPI) relies heavily on structured tables and JSON schemas. Our research suggests these formats are largely invisible to RAG systems. To make API docs "AI-Ready," engineers must generate natural language descriptions alongside the schema definitions.

D. The Agentic Paradox: Retrieval vs. Action

A nuance emerging from this study is the distinc- tion between RAG (Retrieval) and Agents (Action). While our data proves JSON is poor for *retrieval*, it remains the standard for *function calling* by Agents (e.g., OpenAI Functions).

This creates a paradox: documentation needs to be Prose to be found, but JSON to be used. We suggest a dual-layer approach for technical documentation sites: 1. Visible Layer: Natural language descriptions and Q&A for the Vector Index. 2. Hidden Layer: JSON schemas wrapped in HTML <details> tags or separate metadata fields, which are excluded from the embedding process but passed to the LLM context once the document is retrieved.

CONCLUSION AND FUTURE WORK

We have demonstrated that the structure of technical documentation significantly influences its retrievability in RAG systems. Contrary to traditional data modeling best practices, unstructured natural language (specifically Q&A pairs) outperforms structured data (JSON) in vector search tasks.

Future work should investigate the role of Cross-Encoders (Re-ranking models). While this study focused on the bi-encoder retrieval step, it is possible that a powerful re-ranker could rescue the poor performance of JSON content if it reaches the candidate generation stage. Additionally, the impact of Hybrid Search (combining BM25 keywords with Vectors) on these structural findings warrants further exploration. The shift to AI-mediated documentation consumption requires a shift in writing standards.

The era of writing for humans alone is over; we must now write for the vector space they inhabit.

REFERENCES

[1]. P. Aggarwal, V. M. Vishwamitra, and R. R. Shah, "Geo: Generative engine optimization," arXiv preprint arXiv:2311.09735, 2023.

[2]. P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Ku¨ttler, M. Lewis, W.-t. Yih, T. Rockta¨schel, et al., "Retrieval-augmented generation for knowledge- intensive nlp tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[3]. N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, pp. 3982–3992, 2019.

[4]. N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilac- qua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," arXiv preprint arXiv:2307.03172, 2023.

[5]. V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage re- trieval for open-domain question answering," in Proceed- ings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 6769–6781, 2020.

[6]. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for lan- guage understanding," in Proceedings of the 2019 Con- ference of the North American Chapter of the Association for Computational Linguistics, pp. 4171–4186, 2019.

[7]. S. Robertson and H. Zaragoza, "The probabilistic rele- vance framework: Bm25 and beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[8]. K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "Realm: Retrieval-augmented language model pre-training," arXiv preprint arXiv:2002.08909, 2020.

[9]. G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bo- janowski, A. Joulin, and E. Grave, "Unsupervised dense information retrieval with contrastive learning," arXiv preprint arXiv:2112.09118, 2021.

[10]. N. Thakur, N. Reimers, A. Ru¨ckle´, A. Srivastava, and I. Gurevych, "Beir: A heterogeneous benchmark for zero- shot evaluation of information retrieval models," arXiv preprint arXiv:2104.08663, 2021.

[11]. L. Xiong, C. Xiong, Y. Li, K.-F. Tang, J. Liu, P. Bennett, J. Ahmed, and A. Overwijk, "Approximate nearest neigh- bor negative contrastive learning for dense text retrieval," arXiv preprint arXiv:2007.00808, 2020.

[12]. O. Khattab and M. Zaharia, "Colbert: Efficient and effec- tive passage search via contextualized late interaction over bert," in Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 39–48, 2020.

[13]. N. Craswell, "Mean reciprocal rank," in Encyclopedia of database systems, pp. 1703–1703, Springer, 2009.

[14]. B. Hancock, A. Bordes, P.-E. Mazare, and J. Weston, "Learning from dialogue after deployment: Feed yourself, chatbot!," arXiv preprint arXiv:1901.05415, 2019.

[15]. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," Advances in neural information processing systems, vol. 30, 2017.